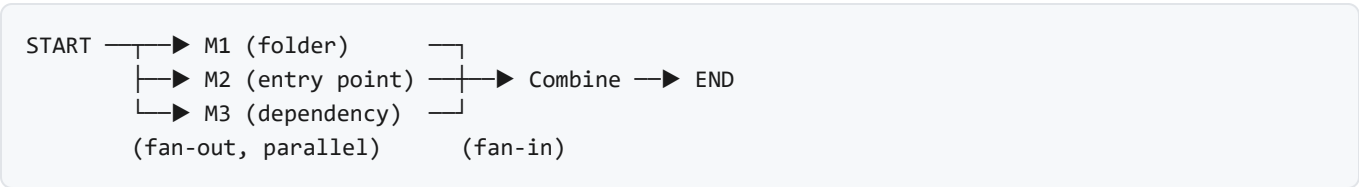


Chapter 04 — AI Pipeline & LangGraph

Component type: Technical — the orchestration engine. **Source:** `backend/src/ai/state.ts` , `model.ts` , `analysis.graph.ts` , `nodes/combine.node.ts` , `backend/src/services/ai.service.ts`

1. Why a graph instead of one big prompt

A single monolithic prompt would suffer context-window saturation, one point of failure, and slow serial reasoning. Instead CodeAstra uses **LangGraph** (`@langchain/langgraph`) to run the three analyses **in parallel** and merge them:



Benefits: **parallelism** (lower latency), **fault isolation** (one node failing doesn't kill the others), and **modularity** (add a new analysis = add a node + two edges).

2. State (`state.ts`)

State is a typed `Annotation.Root` (`GraphState`). Each field declares a **reducer** and a **default**:

Field	Type	Reducer
<code>parsedRepo</code>	<code>ParsedRepo</code>	<code>replace ((_, next) => next), default {files:[], sourcePaths: []}</code>
<code>entryCandidates</code>	<code>string[]</code>	<code>replace, default []</code>
<code>entryContents</code>	<code>{path,content}[]</code>	<code>replace, default []</code>
<code>m1Result</code>	<code>IFolderEntry[] null</code>	<code>replace, default null</code>
<code>m2Result</code>	<code>IEntryPoint null</code>	<code>replace, default null</code>
<code>m3Result</code>	<code>IM3Result null</code>	<code>replace, default null</code>
<code>combined</code>	<code>CombinedOutput null</code>	<code>replace, default null</code>
<code>errors</code>	<code>string[]</code>	<code>accumulate (prev, next) => [...prev, ...next], default []</code>

The one non-destructive field is `errors`. Every node appends its own error string instead of overwriting, so the final state holds the full list of what went wrong across all branches.

3. Models (`model.ts`)

All three models are `ChatOpenAI` instances pointed at NVIDIA's OpenAI-compatible endpoint:

```
BASE_URL = "https://integrate.api.nvidia.com/v1"

M1Model → nvidia/nemotron-3-nano-30b-a3b    temp 0    (fast, simple classification)
M2Model → nvidia/nemotron-3-super-120b-a12b temp 0    (reasoning over entry code)
M3Model → nvidia/nemotron-3-super-120b-a12b temp 0    (structured graph JSON)
```

Keyed by `config.NVIDIA_API_KEY`. `temperature: 0` everywhere → deterministic, analytical output (not creative).

Historical note: earlier docs describe Groq/Llama and Gemini. Those were previous iterations; the current code is Nemotron. See Chapter 00.

4. Structured output (Zod)

Each node binds its model to a Zod schema via `withStructuredOutput(schema)` (e.g. `M1Model.withStructuredOutput(M1OutputSchema)`). The schema fields carry `.describe()` annotations that are fed to the model as inline instructions, which sharply reduces malformed-JSON failures. If the model's output doesn't match the schema, LangChain throws — caught by the node's try/catch.

5. The graph (`analysis.graph.ts`)

```
new StateGraph(GraphState)
  .addNode("m1", M1FolderNode).addNode("m2", m2EntryPointNode)
  .addNode("m3", m3DependencyNode).addNode("combine", combineNode)
  .addEdge(START, "m1").addEdge(START, "m2").addEdge(START, "m3")
  .addEdge("m1", "combine").addEdge("m2", "combine").addEdge("m3", "combine")
  .addEdge("combine", END)
  .compile();
```

`runAnalysisGraph(parsedRepo, entryCandidates, entryContents)` invokes the compiled graph with the initial state and returns `{ result, errors, success }` extracted from the `combined` output. A fatal graph error is caught and returned as `success:false` with `[Graph] Fatal: ...`.

6. Combine node (`combine.node.ts`)

The fan-in. It reads `m1Result / m2Result / m3Result`, pushes a `[Combine] Mx result missing` error for any null one, then assembles:

```
result = {
  m1: m1Result ?? [],
  m2: m2Result ?? { file:"", executionFlow:[], description:"" },
  m3: { graph: m3Result?.graph ?? [], formattedAscii: m3Result?.formattedAscii ?? "" },
};
success = errors.length === 0;
```

So a partial pipeline still yields a valid `IAnalysisResult` — just with `success:false` and warnings.

7. Two-level resilience

1. **Per-node** — every node has try/catch; on failure it returns a safe empty value + an error string. The graph never crashes from one node.
2. **Whole-pipeline fallback** (`ai.service.ts`) — if `runAnalysisGraph` throws or returns `success:false`, the service **synthesizes a result from the AST/regex parse alone** (first ~10 files → M1 guesses by path keywords; first entry candidate → M2; first ~8 files → M3 edges) and returns `success:true`. Net effect: the API almost never returns a hard 500 due to LLM problems — it degrades to a parser-only view.

```
runAnalysisGraph() —success?—► return LLM result
      | no / threw
      ▼
build fallbackResult from parsedRepo (no LLM) —► return success:true (+ warnings)
```

8. Known gaps / improvement ideas

- The AST fallback's M1 `purpose` is keyword-guessed (`route / controller / config`) — coarse.
- No retry/backoff on transient NVIDIA API errors before falling back.
- No token accounting/logging — hard to see cost per analysis.
- Adding a node means editing both `analysis.graph.ts` edges and `state.ts` fields — no registry.